

-0.5cm -0.3cm



Traitement et Analyse de Données

Preprocessing - Cours 1 : Fondamentaux et Exploration

Dr Seydou Nourou SYLLA

Data Scientist Senior
Departement TIC
Universite Alioune Diop de Bambey

January 26, 2026

Overview



1. Introduction

2. Comprendre les données

3. Traitement des données



Introduction

Objectifs du cours

Objectif principal

Maîtriser l'ensemble des techniques de prétraitement des données pour préparer efficacement les données brutes en vue de l'apprentissage automatique.

Objectifs spécifiques

- Comprendre l'importance du preprocessing dans le pipeline de Machine Learning
- Identifier et traiter les problèmes de qualité des données
- Appliquer les techniques de nettoyage, transformation et encodage
- Choisir les méthodes appropriées selon le type de données et l'objectif

Pourquoi le preprocessing est crucial ?

Impact sur les performances

- **Qualité des données** : Garbage in, garbage out – des données de mauvaise qualité produisent de mauvais modèles
- **Performance des modèles** : Le preprocessing peut améliorer la précision de 10-30%
- **Temps de développement** : 60-80% du temps d'un projet Data Science est consacré au preprocessing

Statistique clé

Selon diverses études, le preprocessing représente **60 à 80%** du travail total dans un projet de Machine Learning.

Le preprocessing dans le pipeline ML

Pipeline complet

1. **Collecte** des données brutes
2. **Preprocessing** (nettoyage, transformation, encodage)
3. **Feature Engineering** (création de nouvelles variables)
4. **Modélisation** (entraînement des algorithmes)
5. **Évaluation** et déploiement

Rôle du preprocessing

Transformer les données brutes en un format **propre, cohérent** et **exploitable** par les algorithmes d'apprentissage.

Principales étapes du preprocessing

Les 5 piliers du preprocessing

1. **Gestion des données manquantes** : Imputation, suppression, flags
2. **Nettoyage** : Doublons, valeurs aberrantes, incohérences, bruit
3. **Transformation** : Normalisation, standardisation, transformations mathématiques
4. **Encodage** : Variables catégorielles en format numérique
5. **Enrichissement** : Feature engineering, augmentation des données



Comprendre les données



Information, Donnée, Indicateur : Définitions

Information

- **Définition** : Connaissance ou fait qui a du sens dans un contexte
- **Caractéristique** : Interprétable par l'humain, contextualisée
- **Exemple** : La température est élevée aujourd'hui



Information, Donnée, Indicateur : Définitions

Donnée

- **Définition** : Représentation brute, non interprétée, d'une information
- **Caractéristique** : Format structuré, exploitable par machine
- **Exemple** : 28.5 (degré Celsius)



Information, Donnée, Indicateur : Définitions

Indicateur

- **Définition** : Mesure calculée à partir de données pour évaluer une situation
- **Caractéristique** : Agrégation, comparaison, suivi dans le temps
- **Exemple** : Température moyenne mensuelle : 25°C

Relation : Information $\rightarrow$ Donnée $\rightarrow$ Indicateur

Chaîne de transformation

Information (contexte) $\rightarrow$ **Donnée** (mesure) $\rightarrow$ **Indicateur** (agrégation)

Exemple concret

- **Information** : Il fait chaud
- **Donnée** : température = 35.2°C, date = 2024-07-15, lieu = Dakar
- **Indicateur** : Température moyenne de juillet à Dakar : 32.5°C



Relation : Information $\rightarrow$ Donnée $\rightarrow$ Indicateur

En Machine Learning

On travaille avec des **données** qu'on transforme en **indicateurs** (features) pour créer des modèles qui produisent de nouvelles **informations** (prédictions).

Types de variables : Vue d'ensemble

Classification principale

1. **Qualitatives** (catégorielles) : Décrivent des catégories
2. **Quantitatives** (numériques) : Mesurent des quantités
3. **Autres** : Texte, dates, images, etc.

Pourquoi cette distinction ?

- Chaque type nécessite des traitements spécifiques
- Les algorithmes ML ont des prérequis différents selon le type
- Le preprocessing dépend fortement du type de variable

Variables qualitatives (catégorielles)

Définition

Variables qui décrivent des catégories, des classes ou des attributs non mesurables numériquement.

Sous-types

- **Nominales** : Catégories sans ordre
 - Exemples : Couleur (rouge, bleu, vert), Ville (Paris, Dakar, Lyon)
 - Opérations : Égalité, appartenance à un ensemble
- **Ordinales** : Catégories avec ordre naturel
 - Exemples : Niveau (Débutant, Intermédiaire, Avancé), Satisfaction (Faible, Moyenne, Élevée)
 - Opérations : Comparaison ($<$, $>$), mais pas d'écart mesurable



Variables qualitatives (catégorielles)

Caractéristiques

- Nombre de modalités limité (cardinalité)
- Pas d'opérations arithmétiques possibles
- Nécessitent un encodage pour le ML

Variables quantitatives (numériques)

Définition

Variables qui mesurent des quantités et permettent des opérations mathématiques.

Sous-types

- **Discrètes** : Valeurs entières, comptables
 - Exemples : Nombre d'enfants (0, 1, 2, 3, ...), Nombre de produits achetés
 - Caractéristique : Espacement fini entre valeurs
- **Continues** : Valeurs réelles, mesurables
 - Exemples : Taille (1.75 m), Poids (68.5 kg), Température (23.7°C)
 - Caractéristique : Infinité de valeurs possibles dans un intervalle

Variables quantitatives (numériques)

Opérations possibles

- Toutes les opérations arithmétiques ($+$, $-$, $\times$, $\div$)
- Calculs statistiques (moyenne, écart-type, corrélations)
- Transformations mathématiques (log, sqrt, etc.)



Autres types de variables

Variables textuelles

- **Type** : Chaînes de caractères non structurées
- **Exemples** : Commentaires, descriptions, articles
- **Traitement** : NLP, tokenisation, embeddings

Autres types de variables

Variables temporelles

- **Type** : Dates, timestamps
- **Exemples** : 2024-01-15, 2024-01-15 14:30:00
- **Traitement** : Extraction de features (jour, mois, saison, etc.)

Autres types de variables

Variables complexes

- **Images** : Matrices de pixels, tenseurs
- **Audio** : Signaux temporels, spectrogrammes
- **Géospatiales** : Coordonnées GPS, polygones
- **Graphes** : Structures de relations

Caractéristiques essentielles d'une variable

Les 3 dimensions d'une variable

1. **Type** : Nature de la variable (qualitative, quantitative, etc.)
2. **Format** : Représentation technique (int, float, string, date, etc.)
3. **Rôle** : Fonction dans l'analyse (explicative, cible, identifiante, etc.)

Exemple complet

- **Variable** : age
 - **Type** : Quantitative continue
 - **Format** : float64 (Python) ou REAL (SQL)
 - **Rôle** : Variable explicative (feature)
- UADB • **Domaine** : [0, 120] années

Rôles des variables dans l'analyse

Classification par rôle

- **Variable cible** (target, dépendante) : Ce qu'on cherche à prédire
 - Exemple : Prix d'une maison, classe d'un email (spam/non-spam)
- **Variable explicative** (feature, indépendante) : Utilisée pour prédire
 - Exemple : Surface, nombre de pièces, localisation
- **Variable identifiante** : Identifie de manière unique (ID, clé primaire)
 - Exemple : Numéro de client, identifiant unique
- **Variable de groupement** : Permet de segmenter les données
 - Exemple : Région, catégorie de produit



Rôles des variables dans l'analyse

Important

Le rôle d'une variable peut changer selon l'objectif de l'analyse. Une variable explicative dans un modèle peut devenir la cible dans un autre.

Formats de données : Structures fondamentale

Point (Scalaire)

- **Définition** : Valeur unique, dimension 0
- **Exemple** : `x = 42, nom = "Alice"`
- **Représentation** : Un seul nombre ou une chaîne
- **Usage** : Valeur isolée, paramètre

Vecteur (Array 1D)

- **Définition** : Collection ordonnée de valeurs, dimension 1
- **Exemple** : `[1, 2, 3, 4, 5], np.array([10, 20, 30])`
- **Représentation** : $x = [x_1, x_2, \dots, x_n]^T$
- **Usage** : Une variable pour plusieurs observations

Matrice (Array 2D)

Définition

Tableau à deux dimensions : lignes (observations) et colonnes (variables).

Représentation mathématique

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

où n = nombre d'observations, p = nombre de variables

DataFrame (Tableau structuré)

Définition

Structure tabulaire avec colonnes nommées et types potentiellement différents par colonne.

Caractéristiques

- **Lignes** : Observations (exemples, échantillons)
- **Colonnes** : Variables (features, attributs)
- **Index** : Identifiants des lignes (optionnel)
- **Types mixtes** : Chaque colonne peut avoir un type différent



Liste (List)

Définition

Collection ordonnée et modifiable d'éléments (potentiellement hétérogènes).

Caractéristiques

- **Ordre** : Les éléments sont indexés (0, 1, 2, ...)
- **Modifiable** : Ajout, suppression, modification possibles
- **Hétérogène** : Peut contenir différents types
- **Exemple** : [1, "texte", 3.14, [1, 2]]

Différence avec Array

Les listes Python sont flexibles mais lentes. Les arrays NumPy sont optimisés pour le

calcul numérique.

Dictionnaire (Dict)

Définition

Collection de paires clé-valeur, non ordonnée (avant Python 3.7) ou ordonnée (depuis).

Caractéristiques

- **Accès par clé** : `dict["clé"]` ou `dict.get("clé")`
- **Clés uniques** : Chaque clé apparaît une seule fois
- **Valeurs hétérogènes** : Types peuvent varier
- **Exemple** : `{"nom": "Alice", "age": 25, "notes": [15, 18, 20]}`

Tenseur (Array multidimensionnel)

Définition

Généralisation des matrices à n dimensions.

Dimensions

- **0D** : Scalaire (point)
- **1D** : Vecteur
- **2D** : Matrice
- **3D** : Cube (ex: batch d'images : batch $\times$ hauteur $\times$ largeur)
- **4D** : Hypercube (ex: batch d'images RGB : batch $\times$ hauteur $\times$ largeur $\times$ canaux)

Résumé : Comprendre la donnée

Les 3 niveaux de compréhension

1. **Sémantique** : Information vs Donnée vs Indicateur
2. **Statistique** : Type de variable (qualitative, quantitative, etc.)
3. **Technique** : Structure de données (vecteur, matrice, DataFrame, etc.)

Checklist pour une variable

- Quel est son **type** ? (qualitative/quantitative)
- Quel est son **format** ? (int, float, string, date)
- Quel est son **rôle** ? (cible, explicative, identifiante)
- Quelle est sa **structure** ? (scalaire, vecteur, dans un DataFrame)
- Quelles **opérations** sont possibles ?



Traitement des données

Real world data



Actions

- Il faut constituer un jeu de données de qualité homogène, dont la structure et les formats sont cohérents et bien définis.
- Traiter les valeurs manquantes,
- Supprimer les valeurs aberrantes,
- Redéfinir les échelles des paramètres pour homogénéiser leur variabilité,
- Enrichir les données avec des référentiels externes.

Cette partie représente jusqu'à 80% du travail.

Pourquoi décrire et visualiser avant de nettoyer

Objectifs de l'exploration préalable

- **Comprendre la structure** : Forme, types, distributions
- **Identifier les problèmes** : Valeurs manquantes, aberrations, incohérences
- **Guider le nettoyage** : Choisir les bonnes stratégies selon les patterns observés
- **Établir un état de référence** : Comparer avant/après nettoyage

Principe fondamental

On ne peut pas nettoyer efficacement ce qu'on ne comprend pas. L'exploration visuelle et statistique est **essentielle** avant toute opération de nettoyage.

Description statistique de base

Informations essentielles à collecter

1. **Forme du dataset** : Nombre de lignes, nombre de colonnes
 - `df.shape` en Python, `dim()` en R
2. **Types de données** : Types de chaque colonne
 - `df.dtypes` ou `df.info()`
3. **Statistiques descriptives** : Moyenne, médiane, écart-type, min, max
 - `df.describe()` pour variables numériques
 - `df.value_counts()` pour variables catégorielles

Visualisation des valeurs manquantes

Techniques de visualisation

- **Heatmap des valeurs manquantes** : Identifier les patterns de manque
 - Permet de voir si le manque est aléatoire ou structuré
- **Barplot du pourcentage de manquants** : Par colonne
 - Identifier les colonnes avec trop de manquants ($> 50\%$)
- **Matrice de corrélation des manquants** : Détecter les dépendances
 - Si deux colonnes manquent souvent ensemble, cela suggère un pattern

Visualisation des distributions

Pour variables numériques

- **Histogrammes** : Forme de la distribution (normale, asymétrique, bimodale)
- **Boxplots** : Détecter les valeurs aberrantes (points au-delà des whiskers)
- **Q-Q plots** : Vérifier la normalité
- **Densité (KDE)** : Estimation de la densité de probabilité

Pour variables catégorielles

- **Barplots** : Fréquence de chaque catégorie
- **Pie charts** : Proportions (si peu de catégories)
- **Word clouds** : Pour variables textuelles

Visualisation des valeurs aberrantes

Techniques de détection visuelle

- **Boxplots** : Points au-delà de $Q1 - 1.5 \times IQR$ ou $Q3 + 1.5 \times IQR$
- **Scatter plots** : Points isolés dans l'espace des features
- **Z-score plots** : Points avec $|z| > 3$
- **Isolation Forest visualization** : Points identifiés comme aberrants

Questions à se poser

- Sont-elles des erreurs de saisie ou des valeurs légitimes ?
- Faut-il les supprimer, les transformer ou les imputer ?
- Quel est leur impact sur les statistiques ?

Visualisation des corrélations

Matrice de corrélation

- **Heatmap** : Visualiser les corrélations entre variables numériques
- **Objectifs** :
 - Identifier les variables fortement corrélées (redondance)
 - Détecter les relations linéaires
 - Guider la sélection de features

Scatter plots multiples

- **Pair plots** : Visualiser les relations deux à deux
- Permet de détecter des patterns non-linéaires
- Utile pour identifier des clusters ou des groupes

Visualisation des incohérences

Types d'incohérences à visualiser

- **Formats inconsistants** : Histogramme des formats de dates, casse des textes
- **Valeurs hors domaine** : Points en dehors des plages valides
- **Relations logiques** :
 - Dates incohérentes (achat avant inscription)
 - Âge vs date de naissance
 - Totaux vs sommes partielles

Outils

- **Tableaux croisés** : Pour vérifier les cohérences logiques
- **Graphiques temporels** : Pour détecter les incohérences de dates
- **Validation de règles métier** : Vérifier les contraintes

Checklist d'exploration avant nettoyage

À vérifier systématiquement

1. **Structure** : Forme, types, mémoire utilisée
2. **Valeurs manquantes** : Pourcentage, pattern (MCAR/MAR/MNAR)
3. **Distributions** : Forme, symétrie, normalité
4. **Valeurs aberrantes** : Identification et quantification
5. **Cardinalité** : Nombre de modalités pour variables catégorielles
6. **Corrélations** : Relations entre variables
7. **Incohérences** : Formats, valeurs hors domaine, logique métier
8. **Doublons** : Fréquence et nature

Documentation



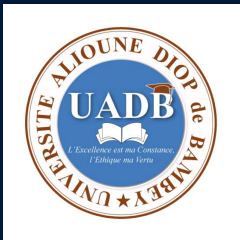
Documentation

Toujours documenter les problèmes identifiés et les décisions prises pour le nettoyage. Cela facilite la reproductibilité et la maintenance.

Outils et bibliothèques recommandées

Python

- **Pandas** : `df.describe()`, `df.info()`, `df.isnull().sum()`
- **Matplotlib/Seaborn** : Visualisations (histogrammes, boxplots, heatmaps)
- **Missingno** : Visualisation spécialisée des valeurs manquantes
- **Sweetviz** : Génération automatique de rapports d'exploration



Thank you for your attention

Dr Seydou Nourou SYLLA

Data Scientist Senior
Departement TIC
Universite Alioune Diop de Bambey

January 26, 2026

Code Python – Description statistique (1/2)

Code Python

```
1 import pandas as pd
2 import numpy as np
3
4 # Forme du dataset
5 print("Shape:", df.shape) # (lignes, colonnes)
6
7 # Types de données
8 print(df.dtypes)
9 df.info()
10
11 # Statistiques descriptives (numériques)
12 print(df.describe())
```

Code Python – Description statistique (2/2)

Code Python

```
1      # Pour variables catégorielles
2      print(df['categorie'].value_counts())
3
4      # Vérifications importantes
5      print("Valeurs manquantes:")
6      print(df.isnull().sum())
7      print("\nCardinalité:")
8      print(df.select_dtypes(include=['object']).nunique())
```

Code Python – Visualisation des valeurs manquantes (1/2)

Code Python

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import missingno as msno
4
5 # Heatmap des valeurs manquantes
6 msno.matrix(df)
7 plt.show()
```

Code Python - Visualisation des valeurs manquantes (2/2)

Code Python

```
1  # Barplot du pourcentage de manquants
2  missing_pct = (df.isnull().sum() / len(df)) * 100
3  missing_pct = missing_pct[missing_pct > 0].sort_values(ascending=
    False)
4  plt.figure(figsize=(10, 6))
5  missing_pct.plot(kind='bar')
6  plt.title("Pourcentage de valeurs manquantes par colonne")
7  plt.show()
```

Code Python – Visualisation des valeurs manquantes (3/3)

Code Python

```
1 # Matrice de corrélation des manquants
2 msno.heatmap(df)
3 plt.title("Corrélation des valeurs manquantes")
4 plt.show()
```

Code Python – Visualisation des distributions (1/

Code Python – Variables numériques

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 from scipy import stats
4
5 # Histogramme
6 df['age'].hist(bins=30, edgecolor='black')
7 plt.title("Distribution de l'âge")
8 plt.show()
9
10 # Boxplot
11 plt.boxplot(df['age'])
12 plt.title("Boxplot de l'âge")
13 plt.show()
```

Code Python – Visualisation des distributions (2)

Code Python – Variables numériques (suite)

```
1  # Q-Q plot pour normalité
2  stats.probplot(df['age'], dist="norm", plot=plt)
3  plt.title("Q-Q Plot")
4  plt.show()
5
6  # Densité (KDE)
7  df['age'].plot(kind='density')
8  plt.title("Densité de l'âge")
9  plt.show()
```

Code Python – Visualisation des distributions (3)

Code Python – Variables catégorielles

```
1  # Barplot
2  df['categorie'].value_counts().plot(kind='bar')
3  plt.title("Fréquence des catégories")
4  plt.show()
5
6  # Pie chart
7  df['categorie'].value_counts().plot(kind='pie', autopct='%1.1f%%')
8  plt.title("Proportions des catégories")
9  plt.show()
```

Code Python – Visualisation des valeurs aberrantes

Code Python

```
1 import numpy as np
2 from scipy import stats
3
4 # Boxplot pour détecter outliers
5 plt.boxplot(df['age'])
6 plt.title("Boxplot - Détection d'outliers")
7 plt.show()
8
9 # Scatter plot
10 plt.scatter(df['age'], df['salaire'])
11 plt.xlabel("Âge")
12 plt.ylabel("Salaire")
13 plt.title("Scatter plot - Outliers visuels")
14 plt.show()
```

Code Python – Visualisation des valeurs aberran

Code Python

```
1 import numpy as np
2 from scipy import stats
3
4
5
6 # Z-score
7 z_scores = np.abs(stats.zscore(df['age']))
8 outliers = df[z_scores > 3]
9 print(f"Nombre d'outliers (|z| > 3): {len(outliers)}")
10
11 # Visualisation Z-score
12 plt.scatter(range(len(df)), z_scores)
13 plt.axhline(y=3, color='r', linestyle='--', label='Seuil z=3')
14 plt.title("Z-scores")
15 plt.ylabel("|Z-score|")
```

Code Python - Visualisation des corrélations (1/2)

Code Python - Matrice de corrélation

```
1 import seaborn as sns
2
3 # Calcul de la matrice de corrélation
4 corr_matrix = df.select_dtypes(include=[np.number]).corr()
5
6 # Heatmap
7 plt.figure(figsize=(10, 8))
8 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm',
9             center=0, fmt='.2f')
10 plt.title("Matrice de corrélation")
11 plt.show()
```

Code Python – Visualisation des corrélations (2/



Code Python

```
1  # Pair plots (scatter plots multiples)
2  numeric_cols = df.select_dtypes(include=[np.number]).columns[:5]
3  sns.pairplot(df[numeric_cols])
4  plt.show()
```

Code Python – Visualisation des incohérences

Code Python

```
1 from datetime import datetime
2
3 # Formats inconsistants - casse
4 print("Valeurs uniques (casse):")
5 print(df['ville'].unique())
6
7 # Valeurs hors domaine
8 invalid_age = df[(df['age'] < 0) | (df['age'] > 120)]
9 print(f"Âges invalides: {len(invalid_age)}")
10
11 # Relations logiques - dates incohérentes
12 df['date_achat'] = pd.to_datetime(df['date_achat'])
13 df['date_inscription'] = pd.to_datetime(df['date_inscription'])
14 incoherent = df[df['date_achat'] < df['date_inscription']]
15 print(f"Dates incohérentes: {len(incoherent)}")
```

Code Python – Visualisation des incohérences

Code Python

```
1 from datetime import datetime
2
3
4
5 # Graphique temporel
6 plt.plot(df['date_achat'], df['montant'], 'o')
7 plt.xlabel("Date d'achat")
8 plt.ylabel("Montant")
9 plt.title("Vérification des incohérences temporelles")
10 plt.show()
11
12 # Tableau croisé
13 pd.crosstab(df['categorie'], df['statut'], margins=True)
```